7 Databases

Databases may seem an unusual topic for a book that deals with coding. But SQL is also code, and AI tools are excellent at compiling SQL commands. In addition, AI is also a great help in other areas, such as the design of databases or the administration of database management systems (DBMSs).

In this chapter, we'll use a wide range of examples to illustrate the use of AI tools in the design or modification of databases, the composition of SQL commands, administration, and finally the development of client code.

DBMS-Specific Answers

Relational database management systems (RDBMSs), such as Microsoft SQL Server, MySQL, Oracle, and PostgreSQL have many things in common, but they differ significantly in the details. Unless you have a very basic question, you should always specify in the prompt which DBMS your question relates to! Otherwise, you may receive answers that don't apply to your DBMS at all.

You'll have the best experience with ChatGPT and others if there is a lot of training material for your DBMS available on the internet. With relational systems, this applies in particular to MySQL/MariaDB, PostgreSQL, SQLite, and SQL Server; with NoSQL products, the very popular programs MongoDB and Redis have an advantage.

7.1 Database Design

As we're going to show you in the following examples, AI tools can save you a lot of work when designing databases and formulating the necessary `CREATE TABLE` commands. As is so often the case, the more precisely you for-

mulate your wishes and the more clearly you can assess which parts of the AI suggestions actually correspond to your requests, the better the AI support will work. If you're not satisfied, you can make improvements through targeted prompts.

7.1.1 Example: Database for an Online Store

Let's assume you want to implement an online store. You pursue a dynamic pricing policy; that is, you want to adjust product prices at any time. So, ask ChatGPT for an initial database design.

Prompt: I want to create a database for an online store. The prices of my products will change frequently (sometimes daily). Can you help me with logical modeling?

> Absolutely, I can help you design a logical model for your online store database. Given that the prices of your products change frequently, we'll need to ensure that the database structure supports efficient updates and historical tracking of price changes.
>
> Here's a basic logical model for your online store database.
>
> **Entities and attributes:**
>
> - Users
> - `UserID (Primary Key)`
> - `Username`
> - `PasswordHash`
> - `Email`
> - `DateRegistered`
> - Categories
> - `CategoryID (Primary Key)`
> - `CategoryName`
> - `Description`
> - ...
>
> **Relationships:**

> - *Users* can place multiple *Orders*.
> - Each *Order* can have multiple *OrderDetails*.
> - . . .
>
> [many more details]

There is little to criticize about the database design suggested by ChatGPT. The organization of the data makes sense, and the number of entities is also reasonable for this design phase. For further discussion of the database design, an entity relationship (ER) diagram would be a good thing to have. Confident as ever, ChatGPT introduces the corresponding request with *Sure*, but then has to admit that it can't draw diagrams. (We're very curious to see when AI tools will be able to do this.)

Prompt: Can you draw an ER diagram for the database?

> Sure, I can describe how the ER diagram for this database would look. Given that I'm unable to directly draw images, I'll provide a detailed description of how you can create the ER diagram. You can then use tools like draw.io, Lucidchart, or any ER diagram tool of your choice to visualize it.
>
> [many more details]

ChatGPT essentially repeats the output of the previous prompt and gives (a few) tips on how this data could be visualized.

But you can outsmart ChatGPT with a trick! There are various tools on the internet for drawing diagrams that expect statements similar to a programming language. This will take ChatGPT back to its text model domain. (You might wonder why ChatGPT doesn't make such a suggestion of its own accord.)

So, ask ChatGPT to create code to draw the ER diagram with Mermaid. Mermaid (**https://mermaid.js.org**) is a great JavaScript library for generating charts and diagrams:

Prompt: Can you provide code to draw an ER diagram with Mermaid?

Sure! Mermaid is a great tool for creating diagrams, including ER diagrams, with code. Below is the Mermaid code to generate the ER diagram for your online store database:

```
erDiagram
    USERS {
        int UserID PK
        string Username
        string PasswordHash
        string Email
        date DateRegistered
    }
    ...
    USERS      ||--o{  ORDERS : "places"
    CATEGORIES ||--o{  PRODUCTS : "contains"
    PRODUCTS   ||--o{  PRODUCT_PRICES : "has"
    ORDERS     ||--o{  ORDER_DETAILS : "includes"
    PRODUCTS   ||--o{  ORDER_DETAILS : "part of"
```

To use this code:

- Copy the Mermaid code above.
- Paste it into a Mermaid live editor.
- The diagram will be rendered automatically.

This Mermaid code defines the entities (tables), their attributes, primary keys (PK), and foreign keys (FK), along with the relationships between

them. The relationships are depicted with the cardinalities: `||--o{` representing a one-to-many relationship.

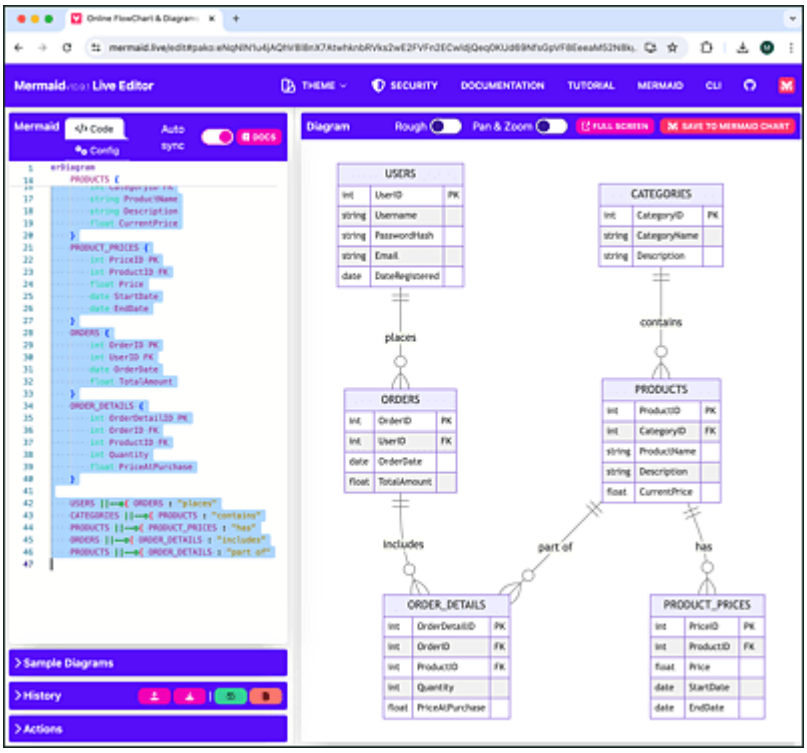

Figure 7.1 The Database Schema as a Mermaid Diagram

This approach works extremely well in a general sense; however, the visual design is certainly open to debate. If required, you can use options to adapt colors, fonts, and design themes to your wishes. If you're not at all happy with Mermaid, you can alternatively ask ChatGPT to formulate the code of the diagram in the syntax of PlantUML, Graphviz, or TikZ.

From the perspective of database theory, ChatGPT can be criticized for not providing an ER diagram at all, but rather a visual representation of the database schema. (An ER diagram is only a preliminary stage. It contains fewer implementation details, for example, no data types such as `int` or `date`. But "real" ER diagrams are much clearer.)

Let's assume you're basically satisfied with the design. The next step is to create the database using `CREATE-TABLE` commands. At this point at the latest, you must decide on a specific DBMS (here: Microsoft SQL Server), and you can also add further entities or columns to the schema, formulate requests for naming the tables and columns, and so on.

Prompt: I would like to implement the database using Microsoft SQL Server. Can you help me with `CREATE TABLE` commands? Also suggest indexes.

Please add some additional columns:

- Customers: first name, last name, address (street, city, etc.)
- Products: URL for product description
- Product prices: VAT rate

Use lowercase names, plural for tables, singular for columns.

> Certainly! Below are the `CREATE TABLE` commands for your database in Microsoft SQL Server, including the additional columns you requested. I've also included suggestions for indexes to optimize query performance.

```
CREATE TABLE users (
    userid INT PRIMARY KEY IDENTITY(1,1),
    username NVARCHAR(50) NOT NULL,
    passwordhash NVARCHAR(255) NOT NULL,
    email NVARCHAR(100) NOT NULL,
    dateregistered DATE NOT NULL,
    firstname NVARCHAR(50),
    lastname NVARCHAR(50),
    street NVARCHAR(100),
    city NVARCHAR(50),
    state NVARCHAR(50),
    zip NVARCHAR(20),
    country NVARCHAR(50)
);
...

-- Indexes
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_categories_categoryname
          ON categories(categoryname);
CREATE INDEX idx_products_categoryid
          ON products(categoryid);
...
```

[explanation: more details]

Warning

It should be clear that even with AI support, you won't be able to quickly design a new database in half an hour. An inadequately thought-out schema will cause major problems as a result. This isn't the fault of the AI tool you're using, but of your insufficiently precise description of the problem.

Of course, this also applies to the example presented here: the `customers` table is missing columns for authentication in the web store (login). The `orders` table lacks information on how payment is made (invoice, PayPal, etc.). The shipping of products (including the calculation of shipping costs) is also not modeled. You usually want to display images with each product. This requires one or two additional tables and considerations as to whether the images should be saved as files or directly in the database. ChatGPT and others are happy to fulfill all these additional requests, but you need to formulate them!

Long story short is that AI tools help you avoid tedious typing. However, you still need to think for yourself about what tasks your database should perform and how best to implement these functions.

7.1.2 Database Management System Selection

In the previous example, we assumed that Microsoft's SQL Server would be used. However, selecting the optimal DBMS is often a topic in itself. AI tools can also help with this, albeit with limitations. In our tests, we've noticed a tendency for AI tools to be reluctant to provide a clear recommendation. Instead, you get a long list of arguments for and against individual products. ChatGPT can't and won't take the decision away from you, but at least it provides you with objective arguments.

The following prompt refers to the online store database discussed earlier.

Prompt: A friend suggested that I use a NoSQL database instead of SQL Server. He mentioned MongoDB. Would this make sense?

> Using a NoSQL database like MongoDB can make sense depending on your specific use case and requirements. Here are some considerations to help you decide whether MongoDB or another NoSQL database is suitable for your online store:
>
> Advantages of Using MongoDB (NoSQL): Schema flexibility, horizontal scalability, document-oriented storage, performance [more details]
>
> Potential Drawbacks of Using MongoDB: Lack of ACID transactions, complex queries, data duplication [more details]
>
> **Example MongoDB Schema for Your Online Store**
>
> Here is an example of how you might structure your data in MongoDB.
>
> [more details]
>
> **Summary**
>
> - Use MongoDB if you need flexible schemas, horizontal scalability, and efficient handling of large volumes of semi-structured or unstructured data.
> - Use SQL Server if your application relies on complex transactions, relationships, and requires strong ACID compliance.
>
> Ultimately, the choice depends on your specific needs, including data structure complexity, scalability requirements, and the nature of your queries and transactions.

ChatGPT lists various advantages and disadvantages of MongoDB compared to SQL Server on approximately two pages of text (reproduced here in abbreviated form). ChatGPT also provides a suggestion for implementing four MongoDB collections to store the data.

After consulting with the team, you decide against MongoDB, but you're unsure as to whether SQL Server is the best choice.

Prompt: After consulting with my team, we decided to use an RDBMS. However, we're considering an open-source product (MariaDB, MySQL,

PostgreSQL) instead. Can you recommend one of these programs?

> All three of the open-source RDBMS options you mentioned (MariaDB, MySQL, PostgreSQL) are robust and widely used, but they have some differences that may make one more suitable for your specific needs. Here's a brief comparison to help you decide:
>
> [more details]
>
> **Summary**
>
> - PostgreSQL is recommended for its advanced features and performance.
> - MariaDB is a strong alternative, especially if you prefer MySQL compatibility and a fully open-source solution.
> - MySQL remains a popular choice for its ease of use and extensive support.
>
> Choose the one that best fits your team's familiarity and the specific needs of your application. If you need further assistance with setting up or configuring your chosen RDBMS, feel free to ask!

7.1.3 Example: Cloud Shopping List

You may be familiar with apps family members use to organize their shopping. If you notice that you're missing milk, flour, or butter, you write the item on a shopping to-do list. Whoever goes shopping next will see the jointly created list and can tick it off again while shopping.

Prompt: We want to build a smartphone app with a cloud backend. In the app, all members of a group (typically a family) can add shopping items. Whoever does the next shopping will see all the items that are currently missing at home. While shopping, he/she can uncheck all items in the shopping cart.

I am responsible for the database backend, which will be implemented using MongoDB. Can you suggest a database schema to handle the shopping list and connect the members of a group?

Certainly! Below is a suggested schema for a MongoDB database to handle a shopping list app where group members can add and manage items.

User collection: Each user has an account and belongs to one or more groups.

```
{
  "_id": "ObjectId",          // Unique identifier for
the user
  "username": "string",       // Username of the user
  "email": "string",          // Email address
  "password": "string",       // Hashed password
  "groups": ["ObjectId"]      // Array of group IDs the
user
}                             // belongs to
```

Group collection: Each group contains multiple users and a list of shopping items.

```
{
  "_id": "ObjectId",          // Unique identifier for
the group
  "name": "string",           // Name of the group
(e.g.,
                              // Family, Friends)
  "members": ["ObjectId"],    // Array of user IDs who
are
                              // members of this group
  "shoppingList": [
    {
      "itemId": "ObjectId",   // Unique identifier for
the item
      "name": "string",       // Name of the item
      "quantity": "string",   // Quantity (2 liters, 1
pack ...)
      "addedBy": "ObjectId", // User ID who added the
item
      "status": "string"      // Status of the item
(pending,
    }                         // purchased ...)
```

```
    ]
}
```

Example Documents: [more details]

Example Queries: [more details]

Key Points:

- **User Authentication:** Store user credentials and provide authentication mechanisms.
- **Group Management:** Each user can belong to multiple groups, and each group can have multiple members.
- **Shopping List Items:** Items in the shopping list have statuses to track whether they are pending or purchased.

ChatGPT's answer goes beyond the actual question. Not only does the AI tool suggest a useful database schema, it also provides concrete examples of how to apply the schema and perform basic database operations (adding items to the shopping list, etc.).

Our earlier warning that you should critically scrutinize ChatGPT's proposed solution naturally also applies here. You probably have detailed requests for the visual and functional design of the app that require corresponding changes to the database schema. But as a starting point for an initial prototype, the answer provided by ChatGPT is perfectly adequate.

7.1.4 Changing the Database Schema

In real life, you rarely have the opportunity to develop a completely new database schema from scratch. More often, you'll only need to expand the schema regarding a few specific details. AI tools can also help in this respect. Typically, in the first step, you transfer the current schema of the database (i.e., all underlying `CREATE TABLE` commands) or make it available in an editor tab. (If required, simply ask ChatGPT how to create a schema file. You can find a corresponding example in **Section 7.3**.)

For some questions, it's sufficient to specify only the definition of a single table. The following question is about how large a text column must be so that IPv6 addresses can be stored without errors in all special cases.

Prompt: MySQL: We use the `logins` table to store IPv4 addresses.

```
CREATE TABLE logins ( ...
  ip VARCHAR(15) DEFAULT NULL,
  ...)
```

Now we have to deal with IPv6 addresses as well. How do we change the `ip` column? Could you provide an `ALTER TABLE` command?

```
ALTER TABLE logins MODIFY COLUMN ip VARCHAR(45)
DEFAULT NULL;
```

This change sets the length of the `ip` column to 45 characters, which is sufficient to store both IPv4 and IPv6 addresses, including the full expanded form of an IPv6 address.

The starting point for the second example is a moderately complex database for storing customer data, orders, and invoices. The schema file comprises approximately 300 lines and contains 10 linked files.

You're working in VS Code, and the schema file is located in your project directory. You want to change the structure of the database. First, you need to make sure that your AI tool understands the structure of the database at all.

Prompt: @workspace Can you explain the content of `accounting-schema.sql`?

The `accounting-schema.sql` file appears to be a MySQL database schema dump for a database named `accounting`. It contains the SQL commands to recreate the structure of the database, including the creation of tables and their relationships. Here are some of the tables defined in the schema . . .

[more details]

Once this test has been completed to your satisfaction, you can formulate what you actually want. Only the contact details of one person can currently be saved in the `customers` table. For some of your customers, however, you want to save several persons.

Prompt: OK. The `customers` table currently contains contact information for only one person. For some customers, we need to store data for several contact persons (first name, last name, email, phone number). Can you suggest a schema change? Existing data must be preserved.

To accommodate multiple contact persons for a single customer, you can create a new table called `contactPersons`. This table will have a foreign key linking it to the `customers` table. Here's the SQL to create this new table:

```
CREATE TABLE contactPersons (
  id          INT(11) NOT NULL AUTO_INCREMENT,
  customerId  INT(11) NOT NULL,
  firstName   VARCHAR(100) DEFAULT NULL,
  lastName    VARCHAR(100) DEFAULT NULL,
  email       VARCHAR(100) NOT NULL,
  telNo       VARCHAR(100) DEFAULT NULL,
  PRIMARY KEY (id),
  FOREIGN KEY (customerId) REFERENCES customers(id)
);
```

In this new schema, each customer can have multiple contact persons. The `customerId` field in the `contactPersons` table links each contact person to a customer in the `customers` table.

To preserve existing data, you can migrate the contact person data from the `customers` table to the `contactPersons` table with a SQL command like this:

```
INSERT INTO contactPersons (customerId, firstName,
                            lastName, email, telNo)
```

```
SELECT id, firstName, lastName, email, telNo
FROM customers;
```

After the data migration, you may want to keep the contact person fields in the `customers` table for a while for backward compatibility. However, you should plan to eventually remove these fields and update any code that references them to use the new `contactPersons` table instead.

We ran the example with GitHub Copilot, and the tool really excelled in this case: not only does it suggest a sensible schema change (outsourcing the contact data to a separate table), it also adopts the column names and data types from the original `customers` table, names the new table in the same camelCase notation that was used for the previous tables, and remembers the new foreign key rule for linking the new table to the existing `customers` table. The suggestions for data migration make sense as well and work without a problem.

Of course, it's still up to you whether you want to carry out the database conversion as suggested or whether changes are still necessary. If you have multiple contacts, you may want to mark one of them as the main contact, in which case, you'll need an additional column.